

Date of Performance: 16 July 2026 **Date of Submission:** 16 July 2026

Devansh Dhawade 24101C0069

Anagha Vichare 24101C0070

Prerna Chavan 24101C0072

Build Data Warehouse/Data Mart for a given problem statement i) Identifying the source tables and populating sample data ii) Design dimensional data model i.e. Star schema, Snowflake schema, and Fact Constellation schema (if applicable).

Practical: Build Data Warehouse/Data Mart for an Omnichannel Retail & Customer Intelligence Hub

Aim To build a Data Warehouse/Data Mart by:

1. Identifying source tables.
2. Populating sample data.
3. Designing dimensional models: o Star Schema o Snowflake Schema o Fact Constellation Schema (Galaxy Schema)

Problem Statement A regional omnichannel retail and e-commerce enterprise wants to build a unified analytics system to model business operations structurally. The existing transactional database stores day-to-day sales records but cannot efficiently generate analytical reports. The management requires reports such as:

- Monthly sales
- Product-wise sales
- Store-wise sales
- Customer purchase trends
- Category-wise revenue Hence, a Sales Data Mart needs to be developed.

Step 1: Identify Source Tables (OLTP Database) Before migrating to the data warehouse, the raw operational data consists of these flat transactional tables:

Customer Table

CustomerID CustomerName Gender City

C101	Amit	Male	Mumbai
C102	Priya	Female	Pune
C103	Rahul	Male	Mumbai
C104	John	Male	New York

Product Table

ProductID ProductName Category Brand Price

P101	Laptop	Electronics	HP	50000
P102	Mobile	Electronics	Samsung	25000
P103	Chair	Furniture	Nilkamal	3000
P104	Desk	Furniture	IKEA	7000

Store Table

StoreID	StoreName	City
S101	Vashi Store	Navi Mumbai
S102	Pune Store	Pune
S103	NYC Hub	New York

Date Table (Transaction)

DateID	Date	Month	Quarter	Year
D001	01-Jan-2026	Jan	Q1	2026
D002	02-Jan-2026	Jan	Q1	2026

Sales Transaction Table

SalesID	DateID	CustomerID	ProductID	StoreID	Quantity
1	D001	C101	P101	S101	1
2	D001	C102	P102	S102	2

Step 2: Populate Sample Data (SQL) – Add 30 records

-- 1. Populate Outer Dimension Tables (The "Snowflakes")

INSERT INTO location_dim (Country_ID, City) VALUES

(1, 'Mumbai'),
(2, 'Pune'),
(3, 'New York');

```
mysql> INSERT INTO location_dim (Country_ID, City) VALUES
-> (1, 'Mumbai'),
-> (2, 'Pune'),
-> (3, 'New York');
Query OK, 3 rows affected (0.495 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM location_dim;
+-----+-----+
| Country_ID | City      |
+-----+-----+
|          1 | Mumbai    |
|          2 | Pune      |
|          3 | New York  |
+-----+-----+
3 rows in set (0.128 sec)
```

INSERT INTO catalog_dim (Catalog_ID, Category) VALUES

(101, 'Electronics'),

(102, 'Furniture');

```
mysql> INSERT INTO catalog_dim (Catalog_ID, Category) VALUES
-> (101, 'Electronics'),
-> (102, 'Furniture');
Query OK, 2 rows affected (0.162 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM catalog_dim;
+-----+-----+
| Catalog_ID | Category    |
+-----+-----+
|          101 | Electronics |
|          102 | Furniture   |
+-----+-----+
2 rows in set (0.011 sec)
```

INSERT INTO month_dim (Month_ID, Month, Quarter, Year) VALUES

(1, 'Jan', 'Q1', 2026),

```
(2, 'Feb', 'Q1', 2026),  
(3, 'Mar', 'Q1', 2026),  
(4, 'Apr', 'Q2', 2026);
```

```
mysql> INSERT INTO month_dim (Month_ID, Month, Quarter, Year) VALUES  
-> (1, 'Jan', 'Q1', 2026),  
-> (2, 'Feb', 'Q1', 2026),  
-> (3, 'Mar', 'Q1', 2026),  
-> (4, 'Apr', 'Q2', 2026);
```

```
Query OK, 4 rows affected (0.042 sec)  
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM month_dim;
```

Month_ID	Month	Quarter	Year
1	Jan	Q1	2026
2	Feb	Q1	2026
3	Mar	Q1	2026
4	Apr	Q2	2026

```
4 rows in set (0.012 sec)
```

-- 2. Populate Inner Dimension Tables

```
INSERT INTO store_dim (StoreID, StoreName, City) VALUES
```

```
('S101', 'Vashi Store', 'Navi Mumbai'),
```

```
('S102', 'Pune Store', 'Pune'),
```

```
('S103', 'NYC Hub', 'New York');
```

```
mysql> INSERT INTO store_dim (StoreID, StoreName, City) VALUES
-> ('S101', 'Vashi Store', 'Navi Mumbai'),
-> ('S102', 'Pune Store', 'Pune'),
-> ('S103', 'NYC Hub', 'New York');
Query OK, 3 rows affected (0.161 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM store_dim;
+-----+-----+-----+
| StoreID | StoreName | City |
+-----+-----+-----+
| S101 | Vashi Store | Navi Mumbai |
| S102 | Pune Store | Pune |
| S103 | NYC Hub | New York |
+-----+-----+-----+
3 rows in set (0.010 sec)
```

INSERT INTO customer_dim (CustomerID, CustomerName, Gender, Country_ID) VALUES

('C101', 'Amit', 'Male', 1),

('C102', 'Priya', 'Female', 2),

('C103', 'Rahul', 'Male', 1),

('C104', 'John', 'Male', 3);

```
mysql> INSERT INTO customer_dim (CustomerID, CustomerName, Gender, Country_ID) VALUES
-> ('C101', 'Amit', 'Male', 1),
-> ('C102', 'Priya', 'Female', 2),
-> ('C103', 'Rahul', 'Male', 1),
-> ('C104', 'John', 'Male', 3);
Query OK, 4 rows affected (0.168 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM customer_dim;
+-----+-----+-----+-----+
| CustomerID | CustomerName | Gender | Country_ID |
+-----+-----+-----+-----+
| C101 | Amit | Male | 1 |
| C102 | Priya | Female | 2 |
| C103 | Rahul | Male | 1 |
| C104 | John | Male | 3 |
+-----+-----+-----+-----+
4 rows in set (0.007 sec)
```

```
INSERT INTO product_dim (ProductID, ProductName, Brand, Price, Catalog_ID) VALUES  
  
('P101', 'Laptop', 'HP', 50000.00, 101),  
  
('P102', 'Mobile', 'Samsung', 25000.00, 101),  
  
('P103', 'Chair', 'Nilkamal', 3000.00, 102),  
  
('P104', 'Desk', 'IKEA', 7000.00, 102);
```

```
mysql> INSERT INTO product_dim (ProductID, ProductName, Brand, Price, Catalog_ID) VALUES  
-> ('P101', 'Laptop', 'HP', 50000.00, 101),  
-> ('P102', 'Mobile', 'Samsung', 25000.00, 101),  
-> ('P103', 'Chair', 'Nilkamal', 3000.00, 102),  
-> ('P104', 'Desk', 'IKEA', 7000.00, 102);  
Query OK, 4 rows affected (0.226 sec)  
Records: 4  Duplicates: 0  Warnings: 0  
  
mysql> SELECT * FROM product_dim;  
+-----+-----+-----+-----+-----+  
| ProductID | ProductName | Brand | Price | Catalog_ID |  
+-----+-----+-----+-----+-----+  
| P101 | Laptop | HP | 50000.00 | 101 |  
| P102 | Mobile | Samsung | 25000.00 | 101 |  
| P103 | Chair | Nilkamal | 3000.00 | 102 |  
| P104 | Desk | IKEA | 7000.00 | 102 |  
+-----+-----+-----+-----+-----+  
4 rows in set (0.010 sec)
```

```
INSERT INTO time_dim (DateID, Date, Month_ID) VALUES  
  
('D001', '2026-01-01', 1),  
  
('D002', '2026-01-02', 1),  
  
('D003', '2026-02-15', 2),  
  
('D004', '2026-03-10', 3);
```

```
mysql> INSERT INTO time_dim (DateID, Date, Month_ID)
-> ('D001', '2026-01-01', 1),
-> ('D002', '2026-01-02', 1),
-> ('D003', '2026-02-15', 2),
-> ('D004', '2026-03-10', 3);
Query OK, 4 rows affected (0.086 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM time_dim;
+-----+-----+-----+
| DateID | Date       | Month_ID |
+-----+-----+-----+
| D001   | 2026-01-01 | 1        |
| D002   | 2026-01-02 | 1        |
| D003   | 2026-02-15 | 2        |
| D004   | 2026-03-10 | 3        |
+-----+-----+-----+
4 rows in set (0.017 sec)
```

-- 3. Populate Central Fact Table (30 Records)

INSERT INTO sales_fact (Transaction_ID, DateID, ProductID, CustomerID, StoreID, Quantity, SalesAmount) VALUES

```
(1, 'D001', 'P101', 'C101', 'S101', 1, 50000.00),
(2, 'D001', 'P102', 'C102', 'S102', 2, 50000.00),
(3, 'D002', 'P103', 'C103', 'S101', 4, 12000.00),
(4, 'D002', 'P104', 'C104', 'S103', 1, 7000.00),
(5, 'D003', 'P101', 'C102', 'S102', 1, 50000.00),
(6, 'D003', 'P102', 'C101', 'S101', 1, 25000.00),
(7, 'D004', 'P103', 'C104', 'S103', 2, 6000.00),
(8, 'D004', 'P104', 'C103', 'S101', 1, 7000.00),
(9, 'D001', 'P103', 'C101', 'S101', 2, 6000.00),
(10, 'D002', 'P102', 'C104', 'S103', 1, 25000.00),
(11, 'D003', 'P104', 'C102', 'S102', 1, 7000.00),
(12, 'D004', 'P101', 'C103', 'S101', 1, 50000.00),
(13, 'D001', 'P104', 'C101', 'S101', 1, 7000.00),
(14, 'D002', 'P101', 'C102', 'S102', 1, 50000.00),
(15, 'D003', 'P103', 'C103', 'S101', 4, 12000.00),
(16, 'D004', 'P102', 'C104', 'S103', 2, 50000.00),
```

(17, 'D001', 'P102', 'C103', 'S101', 1, 25000.00),
 (18, 'D002', 'P104', 'C101', 'S101', 2, 14000.00),
 (19, 'D003', 'P101', 'C104', 'S103', 1, 50000.00),
 (20, 'D004', 'P103', 'C102', 'S102', 3, 9000.00),
 (21, 'D001', 'P101', 'C104', 'S103', 2, 100000.00),
 (22, 'D002', 'P103', 'C102', 'S102', 1, 3000.00),
 (23, 'D003', 'P104', 'C101', 'S101', 1, 7000.00),
 (24, 'D004', 'P102', 'C103', 'S101', 1, 25000.00),
 (25, 'D001', 'P104', 'C102', 'S102', 2, 14000.00),
 (26, 'D002', 'P101', 'C103', 'S101', 1, 50000.00),
 (27, 'D003', 'P103', 'C104', 'S103', 5, 15000.00),
 (28, 'D004', 'P101', 'C101', 'S101', 1, 50000.00),
 (29, 'D001', 'P102', 'C101', 'S101', 1, 25000.00),
 (30, 'D002', 'P103', 'C104', 'S103', 2, 6000.00);

```

mysql> INSERT INTO sales_fact (Transaction_ID, DateID, ProductID, CustomerID, StoreID, Quantity, SalesAmount) VALUES
-> (1, 'D001', 'P101', 'C101', 'S101', 1, 50000.00),
-> (2, 'D001', 'P102', 'C102', 'S102', 2, 50000.00),
-> (3, 'D002', 'P103', 'C103', 'S101', 4, 12000.00),
-> (4, 'D002', 'P104', 'C104', 'S103', 1, 7000.00),
-> (5, 'D003', 'P101', 'C102', 'S102', 1, 50000.00),
-> (6, 'D003', 'P102', 'C101', 'S101', 1, 25000.00),
-> (7, 'D004', 'P103', 'C104', 'S103', 2, 6000.00),
-> (8, 'D004', 'P104', 'C103', 'S101', 1, 7000.00),
-> (9, 'D001', 'P103', 'C101', 'S101', 2, 6000.00),
-> (10, 'D002', 'P102', 'C104', 'S103', 1, 25000.00),
-> (11, 'D003', 'P104', 'C102', 'S102', 1, 7000.00),
-> (12, 'D004', 'P101', 'C103', 'S101', 1, 50000.00),
-> (13, 'D001', 'P104', 'C101', 'S101', 1, 7000.00),
-> (14, 'D002', 'P101', 'C102', 'S102', 1, 50000.00),
-> (15, 'D003', 'P103', 'C103', 'S101', 4, 12000.00),
-> (16, 'D004', 'P102', 'C104', 'S103', 2, 50000.00),
-> (17, 'D001', 'P102', 'C103', 'S101', 1, 25000.00),
-> (18, 'D002', 'P104', 'C101', 'S101', 2, 14000.00),
-> (19, 'D003', 'P101', 'C104', 'S103', 1, 50000.00),
-> (20, 'D004', 'P103', 'C102', 'S102', 3, 9000.00),
-> (21, 'D001', 'P101', 'C104', 'S103', 2, 100000.00),
-> (22, 'D002', 'P103', 'C102', 'S102', 1, 3000.00),
-> (23, 'D003', 'P104', 'C101', 'S101', 1, 7000.00),
-> (24, 'D004', 'P102', 'C103', 'S101', 1, 25000.00),
-> (25, 'D001', 'P104', 'C102', 'S102', 2, 14000.00),
-> (26, 'D002', 'P101', 'C103', 'S101', 1, 50000.00),
-> (27, 'D003', 'P103', 'C104', 'S103', 5, 15000.00),
-> (28, 'D004', 'P101', 'C101', 'S101', 1, 50000.00),
-> (29, 'D001', 'P102', 'C101', 'S101', 1, 25000.00),
-> (30, 'D002', 'P103', 'C104', 'S103', 2, 6000.00);
Query OK, 30 rows affected (0.223 sec)
Records: 30  Duplicates: 0  Warnings: 0
  
```

mysql> SELECT * FROM sales_fact;

Transaction_ID	DateID	ProductID	CustomerID	StoreID	Quantity	SalesAmount
1	D001	P101	C101	S101	1	50000.00
2	D001	P102	C102	S102	2	50000.00
3	D002	P103	C103	S101	4	12000.00
4	D002	P104	C104	S103	1	7000.00
5	D003	P101	C102	S102	1	50000.00
6	D003	P102	C101	S101	1	25000.00
7	D004	P103	C104	S103	2	6000.00
8	D004	P104	C103	S101	1	7000.00
9	D001	P103	C101	S101	2	6000.00
10	D002	P102	C104	S103	1	25000.00
11	D003	P104	C102	S102	1	7000.00
12	D004	P101	C103	S101	1	50000.00
13	D001	P104	C101	S101	1	7000.00
14	D002	P101	C102	S102	1	50000.00
15	D003	P103	C103	S101	4	12000.00
16	D004	P102	C104	S103	2	50000.00
17	D001	P102	C103	S101	1	25000.00
18	D002	P104	C101	S101	2	14000.00
19	D003	P101	C104	S103	1	50000.00
20	D004	P103	C102	S102	3	9000.00
21	D001	P101	C104	S103	2	100000.00
22	D002	P103	C102	S102	1	3000.00
23	D003	P104	C101	S101	1	7000.00
24	D004	P102	C103	S101	1	25000.00
25	D001	P104	C102	S102	2	14000.00
26	D002	P101	C103	S101	1	50000.00
27	D003	P103	C104	S103	5	15000.00
28	D004	P101	C101	S101	1	50000.00
29	D001	P102	C101	S101	1	25000.00
30	D002	P103	C104	S103	2	6000.00

30 rows in set (0.011 sec)

Step 3: Design Star Schema

A Star Schema consists of one central fact table connected directly to multiple dimension tables without any sub-dimension branching.

Customer Dimension

|
|

Product Dimension ---- Fact Sales ---- Store Dimension

|
|

Time Dimension

Fact Table

DateID ProductID CustomerID StoreID Quantity SalesAmount

Measures

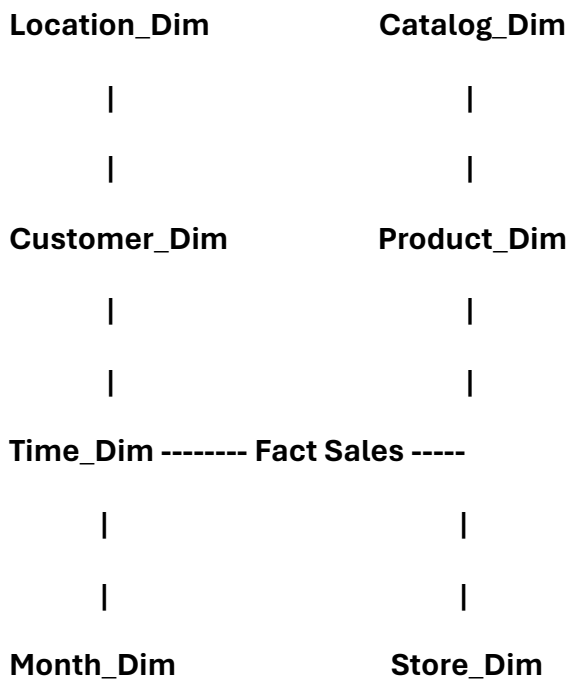
- Quantity Sold
- Sales Amount

Foreign Keys

- DateID
- ProductID
- CustomerID
- StoreID

Step 4: Design Snowflake Schema

A Snowflake Schema normalizes dimension tables into additional sub-dimension tables.



Fact Table

- **sales_fact**: Transaction_ID (PK), DateID (FK), ProductID (FK), CustomerID (FK), StoreID (FK), Quantity, SalesAmount

Normalized Dimension Tables

- **customer_dim** (CustomerID, CustomerName, Gender, Country_ID) links to the outer dimension **location_dim** (Country_ID, City).
- **product_dim** (ProductID, ProductName, Brand, Price, Catalog_ID) links to the outer dimension **catalog_dim** (Catalog_ID, Category).
- **time_dim** (DateID, Date, Month_ID) links to the outer dimension **month_dim** (Month_ID, Month, Quarter, Year).
- **store_dim** (StoreID, StoreName, City) acts as a standard inner dimension connected directly to the fact table

Step 5: Table Creation SQL Queries (Snowflake Schema)

To construct the Snowflake Schema for the Omnichannel Retail & Customer Intelligence Hub, the highly normalized outer dimension tables are created first, followed by the inner dimensions, and finally the central fact table.

-- Outer Dimension Tables

```
CREATE TABLE Location_Dim (  
  
    Country_ID INT PRIMARY KEY,  
  
    City VARCHAR(50)  
  
);
```

```
mysql> CREATE TABLE Location_Dim (  
    ->     Country_ID INT PRIMARY KEY  
    ->     City VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.563 sec)
```

```
CREATE TABLE Catalog_Dim (  
  
    Catalog_ID INT PRIMARY KEY,  
  
    Category VARCHAR(50)  
  
);
```

```
mysql> CREATE TABLE Catalog_Dim (  
    ->     Catalog_ID INT PRIMARY KEY  
    ->     Category VARCHAR(50)  
    -> );  
Query OK, 0 rows affected (0.413 sec)
```

```
CREATE TABLE Month_Dim (  
  
    Month_ID INT PRIMARY KEY,  
  
    Month VARCHAR(20),  
  
    Quarter VARCHAR(10),  
  
    Year INT
```

);

```
mysql> CREATE TABLE Month_Dim (  
->     Month_ID INT PRIMARY KEY,  
->     Month VARCHAR(20),  
->     Quarter VARCHAR(10),  
->     Year INT  
-> );  
Query OK, 0 rows affected (0.334 sec)
```

```
CREATE TABLE Store_Dim (  
  
    StoreID VARCHAR(10) PRIMARY KEY,  
  
    StoreName VARCHAR(50),  
  
    City VARCHAR(50)  
  
);
```

```
mysql> CREATE TABLE Store_Dim (  
->     StoreID VARCHAR(10) PRIMARY KEY,  
->     StoreName VARCHAR(50),  
->     City VARCHAR(50)  
-> );  
Query OK, 0 rows affected (0.392 sec)
```

-- Inner Dimension Tables

```
CREATE TABLE Customer_Dim (  
  
    CustomerID VARCHAR(10) PRIMARY KEY,  
  
    CustomerName VARCHAR(50),  
  
    Gender VARCHAR(10),  
  
    Country_ID INT,  
  
    FOREIGN KEY (Country_ID) REFERENCES Location_Dim(Country_ID)  
  
);
```

```
mysql> CREATE TABLE Customer_Dim (  
->     CustomerID VARCHAR(10) PRIMARY KEY,  
->     CustomerName VARCHAR(50),  
->     Gender VARCHAR(10),  
->     Country_ID INT,  
->     FOREIGN KEY (Country_ID) REFERENCES Location_Dim(Country_ID)  
-> );  
Query OK, 0 rows affected (0.566 sec)
```

```
CREATE TABLE Product_Dim (  
  
    ProductID VARCHAR(10) PRIMARY KEY,  
  
    ProductName VARCHAR(50),  
  
    Brand VARCHAR(50),  
  
    Price DECIMAL(10,2),  
  
    Catalog_ID INT,  
  
    FOREIGN KEY (Catalog_ID) REFERENCES Catalog_Dim(Catalog_ID)  
  
);
```

```
mysql> CREATE TABLE Product_Dim (  
->     ProductID VARCHAR(10) PRIMARY KEY,  
->     ProductName VARCHAR(50),  
->     Brand VARCHAR(50),  
->     Price DECIMAL(10,2),  
->     Catalog_ID INT,  
->     FOREIGN KEY (Catalog_ID) REFERENCES Catalog_Dim(Catalog_ID)  
-> );  
Query OK, 0 rows affected (0.464 sec)
```

```
CREATE TABLE Time_Dim (  
  
    DateID VARCHAR(10) PRIMARY KEY,  
  
    Date DATE,  
  
    Month_ID INT,  
  
    FOREIGN KEY (Month_ID) REFERENCES Month_Dim(Month_ID)
```

);

```
mysql> CREATE TABLE Time_Dim (  
->     DateID VARCHAR(10) PRIMARY KEY,  
->     Date DATE,  
->     Month_ID INT,  
->     FOREIGN KEY (Month_ID) REFERENCES Month_Dim(Month_ID)  
-> );  
Query OK, 0 rows affected (0.643 sec)
```

-- Central Fact Table

```
CREATE TABLE Sales_Fact (  
  
    Transaction_ID INT PRIMARY KEY,  
  
    DateID VARCHAR(10),  
  
    ProductID VARCHAR(10),  
  
    CustomerID VARCHAR(10),  
  
    StoreID VARCHAR(10),  
  
    Quantity INT,  
  
    SalesAmount DECIMAL(12,2),  
  
    FOREIGN KEY (DateID) REFERENCES Time_Dim(DateID),  
  
    FOREIGN KEY (ProductID) REFERENCES Product_Dim(ProductID),  
  
    FOREIGN KEY (CustomerID) REFERENCES Customer_Dim(CustomerID),  
  
    FOREIGN KEY (StoreID) REFERENCES Store_Dim(StoreID)  
  
);
```

```
mysql> CREATE TABLE Sales_Fact (  
->     Transaction_ID INT PRIMARY KEY,  
->     DateID VARCHAR(10),  
->     ProductID VARCHAR(10),  
->     CustomerID VARCHAR(10),  
->     StoreID VARCHAR(10),  
->     Quantity INT,  
->     SalesAmount DECIMAL(12,2),  
->     FOREIGN KEY (DateID) REFERENCES Time_Dim(DateID),  
->     FOREIGN KEY (ProductID) REFERENCES Product_Dim(ProductID),  
->     FOREIGN KEY (CustomerID) REFERENCES Customer_Dim(CustomerID),  
->     FOREIGN KEY (StoreID) REFERENCES Store_Dim(StoreID)  
-> );  
Query OK, 0 rows affected (0.541 sec)
```

Step 6: OLAP Operations

The following OLAP operations were performed on the Sales_Fact table to analyze business performance.

Roll-up:

```
SELECT m.Year, m.Quarter, SUM(sf.SalesAmount) AS TotalSales
```

```
FROM Sales_Fact sf
```

```
JOIN Time_Dim t ON sf.DateID=t.DateID
```

```
JOIN Month_Dim m ON t.Month_ID=m.Month_ID
```

```
GROUP BY m.Year, m.Quarter;
```

```
mysql> SELECT m.Year, m.Quarter, SUM(sf.SalesAmount) AS TotalSales
-> FROM Sales_Fact sf
-> JOIN Time_Dim t ON sf.DateID=t.DateID
-> JOIN Month_Dim m ON t.Month_ID=m.Month_ID
-> GROUP BY m.Year, m.Quarter;
+-----+-----+-----+
| Year | Quarter | TotalSales |
+-----+-----+-----+
| 2026 | Q1      | 796000.00 |
| 2026 | Q2      | 7000.00   |
+-----+-----+-----+
2 rows in set (0.042 sec)
```

Drill-down:

```
SELECT m.Month, SUM(sf.SalesAmount) AS TotalSales
```

```
FROM Sales_Fact sf
```

```
JOIN Time_Dim t ON sf.DateID=t.DateID
```

```
JOIN Month_Dim m ON t.Month_ID=m.Month_ID
```

```
GROUP BY m.Month;
```

```
mysql> SELECT m.Month, SUM(sf.SalesAmount) AS TotalSales
-> FROM Sales_Fact sf
-> JOIN Time_Dim t ON sf.DateID=t.DateID
-> JOIN Month_Dim m ON t.Month_ID=m.Month_ID
-> GROUP BY m.Month;
```

```
+-----+-----+
| Month | TotalSales |
+-----+-----+
| Jan   | 471000.00  |
| Feb   | 165000.00  |
| Mar   | 160000.00  |
| Apr   | 7000.00    |
+-----+-----+
4 rows in set (0.021 sec)
```

Slice:

*SELECT * FROM Sales_Fact WHERE StoreID='S101';*

```
mysql> SELECT * FROM Sales_Fact WHERE StoreID='S101';
+-----+-----+-----+-----+-----+-----+-----+
| Transaction_ID | DateID | ProductID | CustomerID | StoreID | Quantity | SalesAmount |
+-----+-----+-----+-----+-----+-----+-----+
| 1 | D001 | P101 | C101 | S101 | 1 | 50000.00 |
| 3 | D002 | P103 | C103 | S101 | 4 | 12000.00 |
| 6 | D003 | P102 | C101 | S101 | 1 | 25000.00 |
| 8 | D004 | P104 | C103 | S101 | 1 | 7000.00 |
| 9 | D001 | P103 | C101 | S101 | 2 | 6000.00 |
| 12 | D004 | P101 | C103 | S101 | 1 | 50000.00 |
| 13 | D001 | P104 | C101 | S101 | 1 | 7000.00 |
| 15 | D003 | P103 | C103 | S101 | 4 | 12000.00 |
| 17 | D001 | P102 | C103 | S101 | 1 | 25000.00 |
| 18 | D002 | P104 | C101 | S101 | 2 | 14000.00 |
| 23 | D003 | P104 | C101 | S101 | 1 | 7000.00 |
| 24 | D004 | P102 | C103 | S101 | 1 | 25000.00 |
| 26 | D002 | P101 | C103 | S101 | 1 | 50000.00 |
| 28 | D004 | P101 | C101 | S101 | 1 | 50000.00 |
| 29 | D001 | P102 | C101 | S101 | 1 | 25000.00 |
+-----+-----+-----+-----+-----+-----+-----+
15 rows in set (0.010 sec)
```

Dice:

*SELECT * FROM Sales_Fact WHERE ProductID='P101' AND StoreID='S101';*

```
mysql> SELECT * FROM Sales_Fact WHERE ProductID='P101' AND StoreID='S101';
```

Transaction_ID	DateID	ProductID	CustomerID	StoreID	Quantity	SalesAmount
1	D001	P101	C101	S101	1	50000.00
12	D004	P101	C103	S101	1	50000.00
26	D002	P101	C103	S101	1	50000.00
28	D004	P101	C101	S101	1	50000.00

```
4 rows in set (0.008 sec)
```

Pivot:

SELECT ProductID, SUM(SalesAmount) FROM Sales_Fact GROUP BY ProductID;

```
mysql> SELECT ProductID, SUM(SalesAmount) FROM Sales_Fact GROUP BY ProductID;
```

ProductID	SUM(SalesAmount)
P101	500000.00
P102	200000.00
P103	51000.00
P104	52000.00

```
4 rows in set (0.015 sec)
```

Result & Discussion

Result: The Data Warehouse/Data Mart for the Omnichannel Retail system was successfully designed and modeled. Source tables were identified, and sample records were populated to simulate a transactional database. Both a Star Schema and a highly normalized Snowflake Schema were mapped out and supported with Table Creation DDL (Data Definition Language) queries to enable analytical reporting (e.g., Monthly sales, Category-wise revenue).

Discussion: The architectural choice of data model dictates the performance and data integrity of the resulting analytical hub.

Star Schema: This schema was designed to offer extremely fast query performance, making it a highly practical choice for the project, as it avoids complex multi-table JOINS and makes aggregating metrics (like total revenue per customer) highly efficient.

Snowflake Schema: By extending the design into a Snowflake Schema, the dimension tables were normalized (e.g., splitting Customer data from Location data). While this requires more complex JOIN statements that can slightly slow down OLAP operations, it removes text redundancy and ensures strict data integrity.

Ultimately, building this structured data warehouse foundation ensures that any future predictive data mining algorithms run on top of it will benefit from clean, historically accurate data.